Projecte de Laboratori de Programació i Estructures de Dades (PRD): Organitzador de tasques pendents

Aquest projecte de laboratori afronta la implementació en C del nucli d'una senzilla aplicació d'organització de tasques pendents, tipus agenda. L'exercici demana la definició de les estructures de dades necessàries, així com la implementació de les funcions encarregades de proporcionar la funcionalitat desitjada de l'aplicació. A més, també posa èmfasi en la implementació de jocs de proves per a avaluar, de forma progressiva, el bon funcionament de les funcions implementades, sense haver d'esperar a disposar del codi complet de l'aplicació, força més extens i difícilment depurable.

1. Introducció

L'aplicació que desitgem implementar pretén implementar un organitzador de tasques pendents, on l'usuari podrà desar les seves tasques en carpetes per tal d'identificar-les més fàcilment (p.ex., "Família", "Feina", "Lleure", etc.). A més, dins de cada carpeta, les tasques caldrà també organitzar-les en funció de la data en que cal realitzar-les.

A cada tasca creada, l'usuari podrà assignar-li un nivell de prioritat (entre 1 i 3, identificant el valor 1 la prioritat més baixa i 3 la més alta). Així, les tasques d'una certa data, dins d'una carpeta concreta, podran quedar ordenades segons la seva prioritat (en ordre decreixent), permetent a l'usuari visualitzar en primer lloc aquelles tasques pendents més prioritàries. A més, l'usuari podrà afegir comentaris addicionals a les tasques creades, en cas que ho desitgi.

L'aplicació haurà d'oferir una interfície textual d'usuari, capaç de servir les següents opcions introduïdes a través del teclat: `'N'`, `'D'`, `'T'`, `'I'`, `'S'`, `'M'`, `'E'`, `'B'`, `'C'`, `'A'` o `'F'`(en majúscula o minúscula indistintament):

```
Opcions disponibles:
        [N]ova carpeta
        Nova [D]ata
        Nova [T]asca
        [I]nsereix comentari
        Mostra carpete[S]
        [M]ostra carpeta
        [E]limina tasca
        Neteja dates [B]uides
        Elimina [C]arpeta
        [A]juda
        [F]inalitza
```

L'opció `'N'` permetrà crear una nova carpeta, assignant-li un títol. L'opció `'D'` permetrà crear una nova data en una carpeta. L'opció `'T'` permetrà crear una nova tasca, per tal de desar-la en una data d'una certa carpeta, assignant-li un títol i nivell de prioritat. L'opció `'I'` permetrà afegir un comentari a una tasca creada prèviament. L'opció `'S'` permetrà mostrar el títol de totes les carpetes creades a l'aplicació. L'opció `'M'` permetrà mostrar el contingut complet d'una carpeta. L'opció `'E'` permetrà eliminar una tasca creada prèviament. L'opció `'B'` permetrà eliminar totes aquelles dates de qualsevol carpeta que no continguin cap tasca en aquest moment. L'opció `'C'` permetrà eliminar una determinada carpeta. Finalment, les opcions `'A'` i `'F'` permetran mostrar totes les opcions disponibles de l'aplicació i finalitzar l'execució d'aquesta.

2. Disseny de l'aplicació

Un cop analitzats els requisits de l'aplicació que volem desenvolupar, hem cregut oportú estructurar el programa en 4 fitxers de codi font (amb extensió `.c`) i 4 més de capçalera (amb extensió `.h`), tots ells mostrats a continuació:

- `tasca.c`
- `carpeta.c`
- `organitzador.c`
- `main.c`
- `tasca.h`
- `carpeta.h`
- `organitzador.h`
- `main.h`

Dins el fitxer de capçalera `tasca.h` s'hi declaren les estructures necessàries per a representar les tasques gestionades per l'aplicació, així com els comentaris que l'usuari podrà afegir a aquestes. També s'hi declaren totes aquelles funcions que permeten llegir i mostrar les tasques creades i els seus comentaris, a més d'inserir o eliminar comentaris. A `carpeta.h` s'hi declaren les estructures necessàries per a representar les carpetes i dates de l'aplicació. També s'hi declaren funcions per a llegir i mostrar la informació de carpetes i dates, cercar i inserir dates i tasques, eliminar tasques o netejar dates buides d'una carpeta. A `organitzador.h` s'hi declara l'estructura de dades que representa l'organitzador de tasques pendents, a més de funcions per a crear i inserir noves carpetes, mostrar informació de les carpetes o eliminar carpetes. Finalment, a `main.h` s'hi declaren totes aquelles funcions auxiliars encarregades de servir les opcions que la interfície textual posa a disposició de l'usuari. Totes les funcions declarades cal implementar-les als fitxers de codi font corresponents, és a dir, a `tasca.c`, `carpeta.c`, `organitzador.c` i `main.c`. En particular, el bucle principal que implementarà la interfície textual d'usuari de l'aplicació caldrà codificar-lo dins la funció `main()`, al fitxer de codi font `main.c`.

Tractant-se d'una aplicació de complexitat moderada, serà addient comprovar de forma progressiva la correctesa del codi implementat, sense esperar a disposar del seu codi complet, força extens i difícilment depurable. Aleshores, s'afegeixen dos fitxers de capçalera i codi font addicionals, on s'hi declararan i implementaran funcions de test amb jocs de proves diversos:

- `tests.h`
- `tests.c`

Estructures de dades i funcions declarades a tasca.h:

```
#define MAX_C   100     /* Nombre màxim de caràcters  */
#define MIN_PRI 1       /* Valor mínim de prioritat   */
#define MAX_PRI 3       /* Valor màxim de prioritat   */
#define MAX_COM 5       /* Nombre màxim de comentaris */

typedef struct
{
    char text[MAX_C];       /* Text del comentari */
}t_comentari;

typedef struct
{
    char titol[MAX_C];                  /* Títol de la tasca             */
    int prioritat;                      /* Prioritat de la tasca         */
    int ncomentaris;                    /* Nombre de comentaris afegits  */
    t_comentari comentaris[MAX_COM];    /* Comentaris afegits a la tasca */
}t_tasca;
```

```
t_comentari llegeix_nou_comentari();
void mostra_comentari(t_comentari com);
t_tasca llegeix_nova_tasca();
void mostra_tasca(t_tasca tas);
int insereix_comentari(t_tasca *tas, t_comentari com);
int elimina_comentari(t_tasca *tas, int num);
```

La funció `llegeix_nou_comentari()` ha de sol·licitar a l'usuari el text del nou comentari i llegir-lo a continuació (aquest text pot estar format per diverses paraules, si l'usuari ho desitja). Un cop llegit, la funció ha d'inicialitzar una nova variable de tipus `t_comentari` i retornar-la.

La funció `mostra_comentari()` ha de mostrar per pantalla el text del comentari `com`, rebut com a paràmetre.

La funció `llegeix_nova_tasca()` ha de sol·licitar a l'usuari el títol de la nova tasca, així com el seu nivell de prioritat. Mentre l'usuari introdueixi un nivell de prioritat fora del rang `[MIN_PRI, MAX_PRI]`, l'aplicació haurà d'informar-ne i tornar-lo a sol·licitar. Mitjançant el títol i nivell de prioritat finalment llegits, la funció ha d'inicialitzar una nova variable de tipus `t_tasca` (sense cap comentari encara) i retornar-la.

La funció `mostra_tasca()` ha de mostrar per pantalla la informació de la tasca `tas`, rebuda com a paràmetre. És a dir, caldrà que mostri el títol i nivell de prioritat de la tasca, així com els textos de tots els seus comentaris (en cas que en tingui). Podeu consultar l'exemple d'execució esperada inclòs al final d'aquest enunciat per a decidir quin format cal seguir a l'hora de mostrar la informació sol·licitada.

La funció `insereix_comentari()` ha d'inserir el comentari `com` rebut com a paràmetre a la tasca apuntada pel paràmetre `tas`. Aquest nou comentari ha de quedar desat com a primer comentari de l'array de comentaris de la tasca. És a dir, a la posició 0 d'aquest array. En cas que el nou comentari no pugui afegir-se a la tasca (no queda cap posició lliure a l'array de comentaris), la funció ha d'acabar retornant -1. Si el nou comentari pot afegir-se a la tasca de forma satisfactòria, la funció ha d'acabar retornant 0.

La funció `elimina_comentari()` ha d'eliminar el comentari `num` de l'array de comentaris de la tasca apuntada pel paràmetre `tas`. És a dir, ha d'eliminar el comentari que ocupa la posició `num`-1 d'aquest array. En cas que el valor `num` rebut com a paràmetre sigui incorrecte, la funció ha d'acabar retornant -1. Si el comentari pot eliminar-se de la tasca de forma satisfactòria, ha d'acabar retornant 0.

Estructures de dades i funcions declarades a carpeta.h:

```
#include "tasca.h"

#define MAX_TAS 10      /* Nombre màxim de tasques */
#define MAX_DAT 10      /* Nombre màxim de dates   */

typedef struct
{
   int data;                    /* Data representada en format aaaammdd  */
   int ntasques;                /* Nombre de tasques pendents afegides   */
   t_tasca pendents[MAX_TAS];   /* Tasques afegides a la data            */
}t_data;
```

```
typedef struct
{
    char titol[MAX_C];          /* Títol de la carpeta                    */
    int ndates;                 /* Nombre de dates afegides a la carpeta  */
    t_data dates[MAX_DAT];      /* Dates afegides a la carpeta            */
}t_carpeta;

t_data llegeix_nova_data();
void mostra_data(t_data data);
t_carpeta llegeix_nova_carpeta();
void mostra_carpeta(t_carpeta car);
t_data * cerca_data(t_carpeta *car, t_data data);
int insereix_nova_data(t_carpeta *car, t_data data);
int insereix_nova_tasca(t_data *data, t_tasca tas);
t_tasca * cerca_tasca(t_data *data, char titol[MAX_C]);
int elimina_tasca(t_data *data, char titol[MAX_C]);
int neteja_dates_buides(t_carpeta *car);
```

La funció `llegeix_nova_data()` ha de sol·licitar a l'usuari una nova data, en format "`dd/mm/aaaa`". Mitjançant la data introduïda, la funció ha d'inicialitzar una nova variable de tipus `t_data` (sense cap tasca encara) i retornar-la.

La funció `mostra_data()` ha de mostrar per pantalla la informació completa de la data rebuda com a paràmetre. És a dir, cal que mostri la data representada, en format "`dd/mm/aaaa`", així com la informació completa de totes aquelles tasques que guarda.

La funció `llegeix_nova_carpeta()` ha sol·licitar a l'usuari el títol de la nova carpeta i llegir-lo a continuació (aquest títol pot estar format per diverses paraules, si l'usuari ho desitja). Mitjançant el títol llegit, la funció ha d'inicialitzar una nova variable de tipus `t_carpeta` (sense cap data encara) i retornar-la.

La funció `mostra_carpeta()` ha de mostrar per pantalla la informació completa de la carpeta rebuda com a paràmetre. És a dir, cal que mostri el títol de la carpeta, així com la informació completa de totes aquelles dates que guarda.

La funció `cerca_data()` cal que cerqui, dins la carpeta apuntada pel paràmetre `car`, la variable `t_data` que representa la mateixa data que la rebuda com a paràmetre. En cas que la trobi, la funció cal que retorni un punter a aquesta. Si no acaba trobant-la, cal que retorni `NULL`.

La funció `insereix_nova_data()` ha d'inserir la data rebuda com a paràmetre a la carpeta apuntada pel paràmetre `car`. La inserció ha de realitzar-se en aquella posició que mantingui les dates de la carpeta <u>ordenades</u> de forma creixent. És a dir, emmagatzemant aquelles dates més antigues a les primeres posicions de l'array de dates de la carpeta i aquelles més noves a les darreres. En cas que la nova data no pugui inserir-se a la carpeta (no queda cap posició lliure a l'array de dates de la carpeta), la funció ha d'acabar retornant -1. Si la nova data pot inserir-se de forma satisfactòria, la funció ha d'acabar retornant 0.

La funció `insereix_nova_tasca()` ha d'inserir la tasca rebuda com a paràmetre a la data apuntada pel paràmetre `data`. La inserció ha de realitzar-se en aquella posició que mantingui les tasques de la data <u>ordenades</u> de forma decreixent, tenint en compte el seu nivell de prioritat. És a dir, emmagatzemant aquelles tasques més prioritàries a les

primeres posicions de l'array de tasques de la data i aquelles menys prioritàries a les darreres. En cas que la tasca data no pugui inserir-se a la data (no queda cap posició lliure a l'array de tasques de la data), la funció ha d'acabar retornant -1. Si la nova tasca pot inserir-se de forma satisfactòria, la funció ha d'acabar retornant 0.

La funció `cerca_tasca()` cal que cerqui, a la data apuntada pel paràmetre `data`, la tasca amb títol igual al paràmetre `titol`. En cas que la trobi, la funció cal que retorni un punter a aquesta. Si no acaba trobant-la, cal que acabi retornant `NULL`.

La funció `elimina_tasca()` ha d'eliminar, de la data apuntada pel paràmetre `data`, la tasca amb títol igual al paràmetre `titol`. En cas que la trobi i pugui eliminar-la, la funció cal que acabi retornant 0. Si no la troba i no pot eliminar-la, cal que retorni -1.

La funció `neteja_dates_buides()` ha d'eliminar, de la carpeta apuntada pel paràmetre `car`, totes aquelles dates buides (sense cap tasca). La funció ha d'acabar retornant el nombre total de dates buides eliminades.

Estructures de dades i funcions declarades a organitzador.h:

```
#include "carpeta.h"

#define MAX_CAR 5       /* Nombre màxim de carpetes */

typedef struct
{
    int ncarpetes;                  /* Nombre de carpetes creades        */
    t_carpeta carpetes[MAX_CAR];    /* Carpetes creades a l'organitzador */
}t_organitzador;

t_carpeta * cerca_carpeta(t_organitzador *org, char titol[MAX_C]);
int insereix_nova_carpeta(t_organitzador *org, t_carpeta car);
void mostra_carpetes(t_organitzador *org);
void mostra_contingut_carpeta(t_organitzador *org, char titol[MAX_C]);
void neteja_dates_buides_carpetes(t_organitzador *org);
int elimina_carpeta(t_organitzador *org, char titol[MAX_C]);
```

La funció `cerca_carpeta()` cal que cerqui, a l'organitzador apuntat pel paràmetre `org`, la carpeta amb títol igual al paràmetre `titol`. En cas que la trobi, la funció cal que retorni un punter a aquesta. Si no acaba trobant-la, cal que acabi retornant `NULL`.

La funció `insereix_nova_carpeta()` ha d'inserir la carpeta rebuda com a paràmetre a l'organitzador apuntat pel paràmetre `org`. La inserció ha de realitzar-se en aquella posició que mantingui les carpetes <u>ordenades</u> de forma creixent, en funció dels seus títols (ordre alfabètic creixent). En cas que la carpeta no pugui inserir-se a l'organitzador, doncs no queda cap posició lliure a l'array de carpetes de l'organitzador, la funció ha d'acabar retornant -1. Si la nova carpeta pot inserir-se de forma satisfactòria, la funció ha d'acabar retornant 0.

La funció `mostra_carpetes()` ha de mostrar per pantalla el títol de totes les carpetes que guarda l'organizador apuntat pel paràmetre `org`.

La funció `mostra_contingut_carpeta()` ha de mostrar per pantalla la informació completa de la carpeta amb títol igual al paràmetre `titol`, guardada per l'organizador apuntat pel paràmetre `org`. En cas que no trobi la carpeta, ha de mostrar tan sols un missatge per pantalla indicant-ho.

La funció `neteja_dates_buides_carpetes()` ha de netejar les dates buides de totes les carpetes guardades per l'organizador apuntat pel paràmetre `org`. De cada carpeta, cal que la funció en mostri el seu títol i el nombre de carpetes buides eliminades.

La funció `elimina_carpeta()` ha d'eliminar la carpeta amb títol igual al paràmetre `titol` de l'organitzador apuntat pel paràmetre `org`. En cas que la trobi i pugui eliminar-la, la funció cal que acabi retornant 0. Si no la troba i no pot eliminar-la, cal que retorni -1.

Funcions declarades a main.h:

```
#include "organitzador.h"

void mostrar_ajuda();
void llegeix_text(char text[MAX_C]);
void processa_opcio_nova_carpeta(t_organitzador *org);
void processa_opcio_nova_data(t_organitzador *org);
void processa_opcio_mostra_carpetes(t_organitzador *org);
void processa_opcio_nova_tasca(t_organitzador *org);
void processa_opcio_insereix_comentari(t_organitzador *org);
void processa_opcio_mostra_carpeta(t_organitzador *org);
void processa_opcio_elimina_tasca(t_organitzador *org);
void processa_opcio_neteja_dates_buides(t_organitzador *org);
void processa_opcio_elimina_carpeta(t_organitzador *org);
```

Quasi totes les funcions declarades en aquest fitxer de capçalera són funcions auxiliars que caldrà cridar des del bucle principal de la interfície d'usuari, per tal de processar cadascuna de les opcions que l'usuari pot arribar a sol·licitar.

Com a excepció, la funció auxiliar `llegir_text()` s'encarregarà de llegir tan sols una entrada textual de l'usuari (que podrà contenir diverses paraules), dipositant-la a l'array de caràcters `text`, rebut com a paràmetre. Aquesta funció podrà utilitzar-se, per exemple, quan calgui llegir el títol d'una carpeta o d'una tasca, a l'hora de servir les diferents opcions de la interfície d'usuari.

Quan implementeu les funcions `processa_opcio_nova_carpeta()` i `processa_opcio_nova_data()` tingueu en compte que no poden existir múltiples carpetes amb títol idèntic, o diverses dates iguals en una mateixa carpeta.

Podeu consultar l'exemple d'execució esperada de l'aplicació per a decidir com processar cadascuna de les opcions disponibles quan l'usuari les introdueixi.

3. Implementació de l'aplicació

La implementació de l'aplicació serà progressiva, intentant validar mitjançant jocs de proves senzills la correctesa del codi implementat, abans de procedir a la codificació de noves funcions. Amb aquest objectiu, es proposa seguir l'ordre següent:

1. Implementar al fitxer `tasca.c` totes les funcions declarades al fitxer de capçalera `tasca.h`.
2. Validar el correcte funcionament de les funcions implementades prèviament al fitxer `tasca.c`. Amb aquest objectiu, implementar al fitxer `tests.c` les

funcions `void tests_tasca1()` i `void tests_tasca2()`. A la primera funció de test, tan sols cal llegir una nova tasca i mostrar-la, així com un nou comentari i també mostrar-lo. D'aquesta forma podem avaluar la correctesa de les funcions implementades per a llegir i mostrar tasques i comentaris. A la segona funció de test, cal llegir una nova tasca i 3 comentaris diferents. Acte seguit, cal inserir aquests 3 comentaris a la tasca i eliminar-ne un d'ells (p.ex., el comentari número 1). Finalment, cal mostrar el contingut de la tasca, observant-hi tots els comentaris inserits menys l'eliminat. Provar- el codi diverses vegades, eliminant altres comentaris (p.ex., el comentari número 2 o 3). D'aquesta forma, podem avaluar també les funcions que permeten inserir/eliminar comentaris a/de les tasques.

3. Implementar al fitxer `carpeta.c` totes les funcions declarades al fitxer de capçalera `carpeta.h`.
4. Validar el correcte funcionament de les funcions implementades prèviament al fitxer `carpeta.c`. Implementar al fitxer `tests.c` les funcions `void tests_carpeta1()`, `void tests_carpeta2()` i `void tests_carpeta3()`. A la primera funció de test cal llegir una nova carpeta i 3 dates diferents. A continuació cal inserir les 3 dates a la tasca i mostrar-la, observant com les dates s'han inserit correctament, quedant ordenades de forma creixent a la tasca. A la segona funció de test cal llegir una nova carpeta, a més d'una data que cal inserir en aquesta. Acte seguit, cal llegir 3 noves tasques i inserir-les a la data de la carpeta. Finalment, cal mostrar la carpeta, observant com les tasques de la data han quedat ordenades de forma decreixent, segons el seu nivell de prioritat. Proveu també d'eliminar alguna de les tasques de la data abans de mostrar la carpeta, per tal d'assegurar que la funció encarregada d'eliminar tasques funciona correctament. A la darrera funció de test, cal llegir també una nova carpeta, a més de 3 dates que cal inserir en aquesta. A continuació, cal netejar totes les dates buides de la carpeta. Finalment, cal mostrar la carpeta, observant com aquesta s'ha quedat sense cap data, doncs totes les inserides eren buides.
5. Implementar al fitxer `organitzador.c` totes les funcions declarades al fitxer de capçalera `organitzador.h`.
6. Validar el correcte funcionament de les funcions implementades prèviament al fitxer `organitzador.c`. Amb aquest propòsit, implementar al fitxer `tests.c` les funcions `void tests_organizador1()` i `void tests_organitzador2()`. A la primera funció de test cal llegir 3 noves carpetes i inserir-les a una nova variable de tipus `t_organitzador`. Acte seguit, cal mostrar el títol de totes les carpetes inserides, observant com aquestes han quedat ordenades alfabèticament en funció del seu títol. Finalment, cal mostrar el contingut complet d'una de les carpetes inserides usant la funció corresponent per a aquest propòsit. A la segona funció de test cal tornar a llegir 3 noves carpetes i inserir-les a una nova variable de tipus `t_organitzador`. Acte seguit, cal llegir també 2 dates i inserir-les en una de les carpetes. També cal eliminar una de les carpetes buides (sense dates) inserides a l'organitzador, a partir del seu títol, i mostrar el títol de totes les carpetes inserides, observant com ja no hi apareix la carpeta eliminada. Finalment, cal netejar les dates buides de les carpetes de l'organitzador, observant com s'eliminen les dues dates inserides.

7. Implementar al fitxer `main.c` totes les funcions declarades al fitxer de capçalera `main.h`, a més de la funció `main()`, encarregada d'implementar el bucle principal de la interfície textual d'usuari de l'aplicació.
8. Validar el funcionament general de l'aplicació, comprovant que s'ajusta al comportament mostrat a l'exemple de traça d'execució inclós a la secció 4 d'aquest enunciat.

<u>IMPORTANT</u>: Quan lliureu el codi complet de l'aplicació, hi heu d'incloure també les funcions de test que heu implementat per tal de validar-ne parcialment el seu funcionament.

4. Exemple de traça d'execució esperada

A continuació es mostra una traça d'exemple de l'execució esperada de l'aplicació, on les opcions i dades introduïdes per l'usuari es mostren en **<u>negreta</u>**.

```
**********************************************
*      ORGANITZADOR DE TASQUES PENDENTS      *
*   Programacio i Estructures de Dades (PRD) *
*       ETSETB-GREELEC: Curs 2025/2026       *
**********************************************

Opcions disponibles:
        [N]ova carpeta
        Nova [D]ata
        Nova [T]asca
        [I]nsereix comentari
        Mostra carpete[S]
        [M]ostra carpeta
        [E]limina tasca
        Neteja dates [B]uides
        Elimina [C]arpeta
        [A]juda
        [F]inalitza

Introdueix una nova opcio: z
Opcio incorrecta. Torna a intentar-ho.

Introdueix una nova opcio: n
Titol de la nova carpeta: Lleure
Carpeta inserida correctament.

Introdueix una nova opcio: n
Titol de la nova carpeta: Lleure
Ja existeix la carpeta amb aquest titol.

Introdueix una nova opcio: N
Titol de la nova carpeta: Familia
Carpeta inserida correctament.

Introdueix una nova opcio: N
Titol de la nova carpeta: PRD
Carpeta inserida correctament.
```

```
Introdueix una nova opcio: s
Familia
Lleure
PRD

Introdueix una nova opcio: D
Titol de la carpeta: Feina
Carpeta no trobada.

Introdueix una nova opcio: D
Titol de la carpeta: PRD
Data (format: dd/mm/aaaa): 24/04/2026
Data inserida correctament.

Introdueix una nova opcio: T
Titol de la carpeta: Feina
Carpeta no trobada.

Introdueix una nova opcio: T
Titol de la carpeta: PRD
Data (format: dd/mm/aaaa): 20/04/2026
Data no trobada.

Introdueix una nova opcio: T
Titol de la carpeta: PRD
Data (format: dd/mm/aaaa): 24/04/2026
Titol de la nova tasca: Acabar practica de laboratori
Prioritat de la nova tasca (entre [1,3]): 1
Tasca inserida correctament.

Introdueix una nova opcio: T
Titol de la carpeta: PRD
Data (format: dd/mm/aaaa): 24/04/2026
Titol de la nova tasca: Lliurament projecte de laboratori
Prioritat de la nova tasca (entre [1,3]): 5
Valor erroni, torna-hi.
Prioritat de la nova tasca (entre [1,3]): 0
Valor erroni, torna-hi.
Prioritat de la nova tasca (entre [1,3]): 3
Tasca inserida correctament.

Introdueix una nova opcio: I
Titol de la carpeta: PRD
Data (format: dd/mm/aaaa): 24/04/2026
Titol de la tasca: Lliurament projecte de laboratori
Text del nou comentari: Hora maxima de lliurament 23:55h!
Comentari inserit correctament.

Introdueix una nova opcio: I
Titol de la carpeta: PRD
Data (format: dd/mm/aaaa): 24/04/2026
Titol de la tasca: Lliurament projecte de laboratori
Text del nou comentari: Assegurar que el projecte funciona a Netbeans!
Comentari inserit correctament.

Introdueix una nova opcio: D
Titol de la carpeta: PRD
Data (format: dd/mm/aaaa): 08/04/2026
Data inserida correctament.
```

Introdueix una nova opcio: **D**
Titol de la carpeta: **PRD**
Data (format: dd/mm/aaaa): **08/04/2026**
Ja existeix la data a la carpeta.

Introdueix una nova opcio: **T**
Titol de la carpeta: **PRD**
Data (format: dd/mm/aaaa): **08/04/2026**
Titol de la nova tasca: **Examen parcial de teoria**
Prioritat de la nova tasca (entre [1,3]): **3**
Tasca inserida correctament.

Introdueix una nova opcio: **M**
Titol de la carpeta: **PRD**

PRD
Dates:
Data: 08/04/2026
Titol de la tasca: Examen parcial de teoria
Prioritat de la tasca: 3
Data: 24/04/2026
Titol de la tasca: Lliurament projecte de laboratori
Prioritat de la tasca: 3
Comentaris:
1. Assegurar que el projecte funciona a Netbeans!
2. Hora maxima de lliurament 23:55h!
Titol de la tasca: Acabar practica de laboratori
Prioritat de la tasca: 1

Introdueix una nova opcio: **E**
Titol de la carpeta: **Feina**
Carpeta no trobada.

Introdueix una nova opcio: **E**
Titol de la carpeta: **PRD**
Data (format: dd/mm/aaaa): **20/04/2026**
Data no trobada.

Introdueix una nova opcio: **E**
Titol de la carpeta: **PRD**
Data (format: dd/mm/aaaa): **08/04/2026**
Titol de la tasca: **Examen**
Tasca no trobada.

Introdueix una nova opcio: **E**
Titol de la carpeta: **PRD**
Data (format: dd/mm/aaaa): **08/04/2026**
Titol de la tasca: **Examen parcial de teoria**
Tasca eliminada correctament.

Introdueix una nova opcio: **B**
Carpeta: Familia
0 dates eliminades.
Carpeta: Lleure
0 dates eliminades.
Carpeta: PRD
1 dates eliminades.

```
Introdueix una nova opcio: M
Titol de la carpeta: PRD

PRD
Dates:
Data: 24/04/2026
Titol de la tasca: Lliurament projecte de laboratori
Prioritat de la tasca: 3
Comentaris:
1. Assegurar que el projecte funciona a Netbeans!
2. Hora maxima de lliurament 23:55h!
Titol de la tasca: Acabar practica de laboratori
Prioritat de la tasca: 1

Introdueix una nova opcio: C
Titol de la carpeta: Feina
Carpeta no trobada.

Introdueix una nova opcio: C
Titol de la carpeta: Lleure
Carpeta eliminada correctament.

Introdueix una nova opcio: s
Familia
PRD

Introdueix una nova opcio: F
Finalitzant l'aplicacio... Adeu!
```